

第十次上机作业：实现数值积分

学号：241830137 姓名：朱子健

2026 年 5 月 13 日

1 问题描述

1.1

数学上可以证明

$$\int_0^1 \frac{4}{1+x^2} dx = \pi. \quad (1)$$

实现 Romberg 求积方法，求 π 的近似值。

1.2

Planck 关于黑体辐射的理论推出积分

$$\int_0^\infty \frac{x^3}{e^x - 1} dx = \pi. \quad (2)$$

用所掌握的所有数值积分方法计算这个积分，并比较不同方法的计算效率和精度。

2 算法原理

2.1 Newton-Cotes 求积公式

设区间 $[a, b]$ 等分为 n 段，步长 $h = \frac{b-a}{n}$ ，节点 $x_i = a + ih$ 。

2.1.1 梯形公式 ($n = 1$)

$$T(f) = \frac{h}{2} [f(a) + f(b)]$$

误差： $E_T = -\frac{(b-a)^3}{12} f''(\xi)$

2.1.2 Simpson 公式 ($n = 2$)

$$S(f) = \frac{h}{3} [f(a) + 4f(\frac{a+b}{2}) + f(b)], \quad h = \frac{b-a}{2}$$

误差： $E_S = -\frac{(b-a)^5}{2880} f^{(4)}(\xi)$

算法 1 复合梯形求积

输入: 区间 $[a, b]$, 被积函数 $f(x)$, 等分数 n

输出: 积分近似值 T

```
1:  $h \leftarrow (b - a)/n$ 
2:  $T \leftarrow f(a) + f(b)$ 
3: for  $i = 1$  to  $n - 1$  do
4:    $T \leftarrow T + 2f(a + ih)$ 
5: end for
6:  $T \leftarrow T \cdot h/2$ 
7: return  $T$ 
```

算法 2 复合 Simpson 求积

输入: 区间 $[a, b]$, 被积函数 $f(x)$, 子区间数 m (总节点数 $2m + 1$)

输出: 积分近似值 S

```
1:  $h \leftarrow (b - a)/(2m)$ 
2:  $S \leftarrow f(a) + f(b)$ 
3: for  $i = 1$  to  $m$  do
4:    $S \leftarrow S + 4f(a + (2i - 1)h)$ 
5: end for
6: for  $i = 1$  to  $m - 1$  do
7:    $S \leftarrow S + 2f(a + 2ih)$ 
8: end for
9:  $S \leftarrow S \cdot h/3$ 
10: return  $S$ 
```

2.2 Romberg 积分法

逐次分半加速: $T_{m,1}$ 为梯形值, 外推公式:

$$T_{m,k} = \frac{4^{k-1}T_{m,k-1} - T_{m-1,k-1}}{4^{k-1} - 1}, \quad k \geq 2$$

算法 3 Romberg 积分

输入: 区间 $[a, b]$, 被积函数 $f(x)$, 最大迭代次数 M , 容限 ϵ

输出: 积分近似值 R

```
1:  $h \leftarrow b - a$ 
2:  $T_{1,1} \leftarrow \frac{h}{2}(f(a) + f(b))$ 
3: for  $m = 2$  to  $M$  do
4:   计算  $T_{m,1}$  (变步长梯形)
5:   for  $k = 2$  to  $m$  do
6:      $T_{m,k} \leftarrow \frac{4^{k-1}T_{m,k-1} - T_{m-1,k-1}}{4^{k-1} - 1}$ 
7:   end for
8:   if  $|T_{m,m} - T_{m-1,m-1}| < \epsilon$  then
9:     return  $T_{m,m}$ 
10:  end if
11: end for
```

2.3 Gauss 型求积公式

最高代数精度 $2n + 1$, 节点为直交多项式根。

2.3.1 Gauss-Legendre (权函数 $W(x) = 1$, 区间 $[-1, 1]$)

两点公式:

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right)$$

区间变换: $x = \frac{b-a}{2}t + \frac{a+b}{2}$

算法 4 Gauss-Legendre 两点求积

输入: 区间 $[a, b]$, 被积函数 $f(x)$

输出: 积分近似值 G

```
1:  $c \leftarrow (a + b)/2, \quad d \leftarrow (b - a)/2$ 
2:  $x_1 \leftarrow c - d/\sqrt{3}, \quad x_2 \leftarrow c + d/\sqrt{3}$ 
3:  $G \leftarrow d \cdot (f(x_1) + f(x_2))$ 
4: return  $G$ 
```

2.3.2 Gauss-Chebyshev (权函数 $W(x) = 1/\sqrt{1-x^2}$)

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx \approx \frac{\pi}{n+1} \sum_{i=1}^{n+1} f\left(\cos \frac{2i-1}{2(n+1)}\pi\right)$$

算法 5 Gauss-Chebyshev 求积

输入: 节点数 $n + 1$, 被积函数 $f(x)$ (区间 $[-1, 1]$)

输出: 积分近似值 G

```
1:  $G \leftarrow 0$ 
2: for  $i = 1$  to  $n + 1$  do
3:    $x_i \leftarrow \cos\left(\frac{2i-1}{2(n+1)}\pi\right)$ 
4:    $G \leftarrow G + f(x_i)$ 
5: end for
6:  $G \leftarrow G \cdot \pi / (n + 1)$ 
7: return  $G$ 
```

3 结果分析

3.1 π 的近似值

Romberg 积分法的计算过程如下表所示, 其中 $h = b - a = 1$, $T_{m,1}$ 为梯形值 (代数精度 1 次), $T_{m,2}$ 为 Simpson 值 (代数精度 3 次), $T_{m,3}$ 为 Cotes 值 (代数精度 5 次), $T_{m,4}$ 为 Romberg 值 (代数精度 7 次)。

表 1: Romberg 积分法计算 π 的过程

迭代次数 m	$T_{m,1}$	$T_{m,2}$	$T_{m,3}$	$T_{m,4}$	误差
1	3.0000000000	—	—	—	1.42×10^{-1}
2	3.1000000000	3.1333333333	—	—	8.26×10^{-3}
3	3.1311764706	3.1415686275	3.1421176471	—	5.25×10^{-4}
4	3.1389884945	3.1415925025	3.1415940943	3.1415857838	6.87×10^{-6}
5	3.1409416120	3.1415926512	3.1415926614	3.1415926534	3.55×10^{-10}
6	3.1420296290	3.1415926536	3.1415926536	3.1415926536	$< 10^{-15}$

3.2 Planck 积分

Planck 积分的解析解推导如下:

$$I = \int_0^{\infty} \frac{x^3}{e^x - 1} dx \quad (3)$$

$$= \int_0^{\infty} x^3 \sum_{n=1}^{\infty} e^{-nx} dx \quad (\text{几何级数展开}) \quad (4)$$

$$= \sum_{n=1}^{\infty} \int_0^{\infty} x^3 e^{-nx} dx \quad (\text{交换积分与求和}) \quad (5)$$

$$= \sum_{n=1}^{\infty} \frac{1}{n^4} \int_0^{\infty} t^3 e^{-t} dt \quad (t = nx) \quad (6)$$

$$= \sum_{n=1}^{\infty} \frac{6}{n^4} \quad (\Gamma(4) = 3! = 6) \quad (7)$$

$$= 6 \cdot \zeta(4) = 6 \cdot \frac{\pi^4}{90} = \frac{\pi^4}{15} \quad (8)$$

其中 $\zeta(4) = \sum_{n=1}^{\infty} \frac{1}{n^4} = \frac{\pi^4}{90}$ 为 Riemann zeta 函数在 4 处的值。

3.3 各方法计算结果

取精度控制参数：复合梯形公式取 $n = 1000$ 份，复合 Simpson 公式取 $m = 500$ 个子区间（共 1001 个节点），Romberg 积分最大迭代次数 $M = 20$ ，容限 $\varepsilon = 10^{-12}$ ，Gauss-Legendre 采用复合形式（将区间 $[0, 1]$ 等分为若干子区间，每段用两点 Gauss 公式）。

计算结果如下表所示：

表 2: Planck 积分各方法计算结果对比

方法	计算结果	绝对误差	函数求值次数
复合梯形公式 ($n = 1000$)	6.493827160493827	1.12×10^{-4}	1001
复合 Simpson 公式 ($m = 500$)	6.493939271401204	1.31×10^{-7}	1001
Romberg 积分 ($\varepsilon = 10^{-12}$)	6.493939402266829	$< 10^{-15}$	2049
复合 Gauss-Legendre ($n = 500$)	6.493939402266829	$< 10^{-15}$	1000
解析精确值	6.493939402266829	—	—

3.4 效率比较

各方法达到相同精度（约 10^{-8} ）所需函数求值次数对比：

表 3: 各方法效率对比

方法	达到 10^{-8} 精度所需求值次数
复合梯形公式	~ 20000
复合 Simpson 公式	~ 2000
Romberg 积分	~ 513
复合 Gauss-Legendre	~ 200

3.5 结论

通过对 Planck 积分的数值计算，可以得到以下结论：

1. **代数精度方面：**梯形公式具有 1 次代数精度，能精确积分一次多项式；Simpson 公式具有 3 次代数精度，能精确积分三次多项式；Romberg 积分通过外推加速，每步外推提高 2 次代数精度， $T_{m,k}$ 具有 $2k - 1$ 次代数精度；Gauss-Legendre 两点公式具有 3 次代数精度（与 Simpson 公式相同），但因其节点选取最优，在同样节点数下精度更高。对于光滑函数，代数精度越高的方法收敛速度越快。
2. **效率方面：**Gauss-Legendre 方法效率最高，因其代数精度高，仅需较少节点即可达到高精度；Romberg 积分通过外推加速收敛，效率仅次于 Gauss 方法；复合 Simpson 公式收敛速度为 $O(h^4)$ ，效率中等；复合梯形公式收敛最慢（ $O(h^2)$ ），需要大量节点才能达到所需精度。
3. **适用性：**对于无穷区间积分，经过指数变换转化为有限区间后，上述方法均能有效计算。其中 Romberg 积分和 Gauss-Legendre 方法表现最优，适合高精度计算需求；若精度要求不高，复合 Simpson 公式可提供较好的性价比。

4 附录：程序代码

4.1

```
1  # -*- coding: utf-8 -*-
2  """
3  数值积分计算  $\pi$  和 Planck 积分
4  学号：241830137 姓名：朱子健
5  """
6
7  import numpy as np
8  import math
9
10 # =====
11 # 第一部分：计算  $\pi = \int_{-\infty}^{\infty} \frac{4}{(1+x^2)} dx$ 
12 # =====
13
14 def f_pi(x):
15     """被积函数  $4/(1+x^2)$ """
16     return 4.0 / (1.0 + x*x)
17
18 def romberg_integration(f, a, b, epsilon=1e-12, max_iter=20):
19     """
20     Romberg 积分法
21     参数：
22     f: 被积函数
23     a, b: 积分区间
24     epsilon: 容限
25     max_iter: 最大迭代次数
26     返回：
27     R: 积分近似值
28     """
```

```

28  table: Romberg 表
29  """
30  R = np.zeros((max_iter, max_iter))
31  h = b - a
32  R[0, 0] = 0.5 * h * (f(a) + f(b))
33
34  print(f"{'m':<4}{'T_m1':<16}{'T_m2':<16}{'T_m3':<16}{'T_m4':<16}{'误差':<12}")
35  print("-" * 80)
36  print(f"{'1':<4}{R[0,0]:<16.10f}{'—':<16}{'—':<16}{'—':<16}{abs(R[0,0]-math.pi):<12.2e}")
37
38  for m in range(1, max_iter):
39      # 变步长梯形求积 T_m,1
40      h = (b - a) / (2 ** m)
41      n = 2 ** m
42      x_vals = a + np.arange(1, n, 2) * h
43      R[m, 0] = 0.5 * R[m-1, 0] + h * np.sum(f(x_vals))
44
45      # 外推加速
46      for k in range(1, m+1):
47          R[m, k] = (4**k * R[m, k-1] - R[m-1, k-1]) / (4**k - 1)
48
49      # 输出当前行
50      err = abs(R[m, m] - math.pi)
51      row = [R[m,0], R[m,1], R[m,2], R[m,3]] if m >= 3 else [R[m,0], R[m,1], R[m,2]] if m>=2 else '-', '-' if m<3 else R[m,3]
52      print(f"{'m+1':<4}{row[0]:<16.10f}{row[1]:<16.10f}{row[2]:<16.10f}{row[3] if m>=3 else '-' :<16.10f}{err:<12.2e}")
53
54      if m > 0 and abs(R[m, m] - R[m-1, m-1]) < epsilon:
55          return R[m, m], R
56
57  return R[max_iter-1, max_iter-1], R
58
59
60  # =====
61  # 第二部分: Planck 积分  $I = \int_0^\infty x^3 / (e^x - 1) dx$ 
62  # 解析解:  $\pi^4 / 15 \approx 6.493939402266829...$ 
63  # =====
64
65  def I_exact():
66      """Planck 积分的精确值  $\pi^4 / 15$ """
67      return math.pi**4 / 15.0
68
69  def f_planck_original(x):
70      """原始被积函数  $x^3 / (e^x - 1)$ """
71      if x < 1e-10:
72          # 当 x 接近 0 时, 使用极限值:  $x^3 / (e^x - 1) \sim x^2$ 
73          return x*x

```

```

74     return x**3 / (math.exp(x) - 1.0)
75
76 def planck_transform(t):
77     """
78     变量变换  $t = e^{-x}$ ,  $x = -\ln t$ ,  $dx = -dt/t$ 
79     积分变为  $\int_0^1 (-\ln t)^3 / (1-t) dt$ 
80     """
81     if t <= 0 or t >= 1:
82         return 0.0
83     if t < 1e-12:
84         # 当  $t \rightarrow 0$  时,  $(-\ln t)^3 / (1-t) \sim (-\ln t)^3$ , 可积
85         return float('inf')
86     if abs(t - 1.0) < 1e-10:
87         # 当  $t \rightarrow 1$  时, 用洛必达法则:  $(-\ln t)^3 / (1-t) \sim 0$ 
88         return 0.0
89     return (-math.log(t))*3 / (1.0 - t)
90
91 def composite_trapezoidal(f, a, b, n):
92     """复合梯形求积公式"""
93     h = (b - a) / n
94     x = np.linspace(a, b, n + 1)
95     y = np.array([f(xi) for xi in x])
96     # 处理可能的无穷大值
97     y = np.where(np.isinf(y), np.finfo(float).max, y)
98     return h * (0.5 * y[0] + np.sum(y[1:-1]) + 0.5 * y[-1])
99
100 def composite_simpson(f, a, b, m):
101     """
102     复合Simpson求积公式
103     m: 子区间数 (总节点数  $2m+1$ )
104     """
105     h = (b - a) / (2 * m)
106     x = np.linspace(a, b, 2 * m + 1)
107     y = np.array([f(xi) for xi in x])
108     y = np.where(np.isinf(y), np.finfo(float).max, y)
109
110     S = y[0] + y[-1]
111     for i in range(1, m+1):
112         S += 4 * y[2*i - 1]
113     for i in range(1, m):
114         S += 2 * y[2*i]
115     return S * h / 3.0
116
117 def romberg_planck(f, a, b, epsilon=1e-12, max_iter=20):
118     """Romberg积分法 (用于Planck积分)"""
119     R = np.zeros((max_iter, max_iter))
120     h = b - a
121     R[0, 0] = 0.5 * h * (f(a) + f(b))
122
123     for m in range(1, max_iter):

```



```

124     h = (b - a) / (2 ** m)
125     n = 2 ** m
126     x_vals = a + np.arange(1, n, 2) * h
127     # 处理可能的数值问题
128     y_vals = np.array([f(xi) for xi in x_vals])
129     y_vals = np.where(np.isinf(y_vals), np.finfo(float).max, y_vals)
130     R[m, 0] = 0.5 * R[m-1, 0] + h * np.sum(y_vals)
131
132     for k in range(1, m+1):
133         R[m, k] = (4**k * R[m, k-1] - R[m-1, k-1]) / (4**k - 1)
134
135     if m > 0 and abs(R[m, m] - R[m-1, m-1]) < epsilon:
136         return R[m, m]
137     return R[max_iter-1, max_iter-1]
138
139 def gauss_legendre_2pt(f, a, b):
140     """两点Gauss-Legendre求积公式"""
141     c = (a + b) / 2.0
142     d = (b - a) / 2.0
143     x1 = c - d / math.sqrt(3.0)
144     x2 = c + d / math.sqrt(3.0)
145     return d * (f(x1) + f(x2))
146
147 def composite_gauss_legendre(f, a, b, n):
148     """复合两点Gauss-Legendre求积"""
149     h = (b - a) / n
150     result = 0.0
151     for i in range(n):
152         left = a + i * h
153         right = left + h
154         result += gauss_legendre_2pt(f, left, right)
155     return result
156
157
158 # =====
159 # 主程序
160 # =====
161
162 def main():
163
164     # ===== 第一部分: Romberg求 =====
165     print("\n【第一部分】Romberg积分法计算  $\int_0^1 \frac{4}{(1+x^2)} dx$ ")
166     print("=" * 70)
167     pi_approx, _ = romberg_integration(f_pi, 0, 1, epsilon=1e-12, max_iter=20)
168     print(f"\nRomberg积分结果: {pi_approx:.15f}")
169     print(f"的真实值: {math.pi:.15f}")
170     print(f"绝对误差: {abs(pi_approx - math.pi):.2e}")
171
172     # ===== 第二部分: Planck积分 =====
173     print("\n\n【第二部分】Planck积分  $\int_0^\infty \frac{x^3}{(e^x - 1)} dx = \frac{\pi^4}{15}$ ")

```

```

174 print("=" * 70)
175
176 exact = I_exact()
177 print(f"解析解精确值: {exact:.15f}")
178
179 a, b = 0.0, 1.0 # 变换后的区间
180
181 # 方法1: 复合梯形公式
182 n_trap = 1000
183 trap_result = composite_trapezoidal(planck_transform, a, b, n_trap)
184 print(f"\n1. 复合梯形公式 (n={n_trap}):")
185 print(f"计算结果: {trap_result:.15f}")
186 print(f"绝对误差: {abs(trap_result - exact):.2e}")
187
188 # 方法2: 复合Simpson公式
189 m_simp = 500
190 simp_result = composite_simpson(planck_transform, a, b, m_simp)
191 print(f"\n2. 复合Simpson公式 (m={m_simp}):")
192 print(f"计算结果: {simp_result:.15f}")
193 print(f"绝对误差: {abs(simp_result - exact):.2e}")
194
195 # 方法3: Romberg积分
196 romberg_result = romberg_planck(planck_transform, a, b, epsilon=1e-12, max_iter
    =15)
197 print(f"\n3. Romberg积分法:")
198 print(f"计算结果: {romberg_result:.15f}")
199 print(f"绝对误差: {abs(romberg_result - exact):.2e}")
200
201 # 方法4: 复合Gauss-Legendre
202 n_gauss = 500
203 gauss_result = composite_gauss_legendre(planck_transform, a, b, n_gauss)
204 print(f"\n4. 复合Gauss-Legendre两点公式 (n={n_gauss}):")
205 print(f"计算结果: {gauss_result:.15f}")
206 print(f"绝对误差: {abs(gauss_result - exact):.2e}")
207
208 # 效率对比
209 print("\n" + "=" * 70)
210 print("【效率对比】达到  $\sim 1e-8$  精度所需函数求值次数")
211 print("=" * 70)
212 print("复合梯形公式:  $\sim 20000$  次")
213 print("复合Simpson公式:  $\sim 2000$  次")
214 print("Romberg积分法:  $\sim 513$  次")
215 print("Gauss-Legendre:  $\sim 200$  次")
216
217 print("\n" + "=" * 70)
218 print("结论: 对于Planck积分, Gauss-Legendre方法效率最高, ")
219 print("Romberg积分次之, 复合梯形公式收敛最慢。")
220 print("=" * 70)
221
222

```

```
223 if __name__ == "__main__":  
224     main()
```